SOFTWARE REQUIREMENTS SPECIFICATION

FOR

Website Vulnerability Scanner

Prepared by

Deepthi K (241IT021)

Jasmine (241IT051)

Sacheth Koushal (241IT058)

DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA SURATHKAL

Version 1.0

31 July 2026

Table of Contents

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the **Website Vulnerability Scanner (WVS)**, a web-based application that performs automated security assessment of publicly reachable web applications and presents the results through an interactive dashboard. It is written for project supervisors and examiners, as the authoritative statement of what the system shall do; for the development team, as the input to design, implementation, and testing; and for future maintainers.

This document describes *what* the system shall do, not *how*, except where an implementation choice is itself a requirement (§3.4).

1.2 Scope

WVS accepts a target web application identified by URL, verifies that the requesting user is authorised to test that target, crawls the target to discover its reachable surface, executes a catalogue of vulnerability detectors, and produces a severity-ranked set of findings with supporting evidence and remediation guidance. Results are viewable live in a browser dashboard and exportable as PDF, HTML, JSON, CSV, and SARIF reports.

WVS is a **detection** tool, not an exploitation tool. It does not:

- Exploit discovered vulnerabilities (shell access, data extraction, privilege escalation)
- Scan behind a login session (authenticated scanning is deferred to a future release)
- Perform denial-of-service, stress, or load testing
- Brute-force credentials
- Use blind or time-based injection techniques
- Scan non-web ports, or native mobile/desktop software
- Apply remediation automatically; it reports, and humans remediate

Legal and ethical position. Unauthorised scanning of computer systems is a criminal offence in most jurisdictions (Information Technology Act 2000, CFAA, Computer Misuse Act 1990). WVS is for use **only** against targets the user owns or has written permission to test. The authorisation controls in §3.1 (F.2) are mandatory requirements and shall not be bypassable through configuration. During development and demonstration, scanning is permitted only against the project's own vulnerable fixture application, locally hosted intentionally-vulnerable training applications, or targets with documented written authorisation.

1.3 Definitions, Acronyms, and Abbreviations

- **Target** – A web application, identified by an origin (scheme + host + port), registered in the system for scanning.
- **Scan** – A single bounded execution of the scan engine against one target, producing zero or more findings.
- **Detector** – A self-contained module identifying one class of vulnerability or misconfiguration.
- **Finding** – A single detected issue: a detector identifier, location, evidence, severity, and remediation guidance.
- **Evidence** – The retained request/response data substantiating a finding, so a human can independently verify it.

- **Crawl** – The discovery phase enumerating reachable URLs, forms, and parameters within scope.
- **Scope** – The rules determining which URLs a scan may and may not request.
- **Passive check** – A detector drawing conclusions only from traffic the scanner would generate anyway; no crafted payloads.
- **Safe active check** – A detector sending crafted but non-destructive requests: idempotent methods, benign payloads.
- **Ownership verification** – The process by which a user demonstrates administrative control of a target before scanning is permitted.
- **`CONFIRMED`** – Confidence level: the scanner directly observed the vulnerable behaviour, for example its marker reflected unencoded into an executable context.
- **`FIRM`** – Confidence level: strong indirect evidence, but the vulnerable behaviour was not directly triggered.
- **`TENTATIVE`** – Confidence level: a pattern or version match consistent with the weakness; independent verification required.

Acronyms: CORS, CSP, CSRF, CVE, CVSS, CWE, DAST (Dynamic Application Security Testing), EPSS (Exploit Prediction Scoring System), HSTS, OSV (Open Source Vulnerabilities database), OWASP, SARIF, SQLi, SSRF, TLS, XSS.

Notation: `F.n`: functional requirement; `NFR-CAT-n`: non-functional requirement (CAT: PERF, SEC, USE, REL, MAINT, SCAL, PORT, QUAL); `C.n`: project constraint (§2.4); `DC-n`: design constraint (§3.4). **Shall** = mandatory, **should** = recommended, **may** = optional (RFC 2119). Priorities: **M** = must (absence is a project failure), **S** = should (omission needs justification), **C** = could (if schedule permits).

1.4 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
2. OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks.
3. OWASP Web Security Testing Guide (WSTG), v4.2.
4. FIRST, *Common Vulnerability Scoring System v3.1: Specification Document*.
5. MITRE, *Common Weakness Enumeration (CWE)*.
6. OASIS, *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*.
7. Open Source Vulnerabilities (OSV) database schema and API.
8. FIRST, *Exploit Prediction Scoring System (EPSS) Model and API Specification*.

1.5 Overview

Section 2 describes the product in context. Section 3 lists the numbered, testable requirements, which are the core of this document. Section 4 contains supporting catalogues and reference material.

---

2. Overall Description

2.1 Product Perspective

WVS is a new, self-contained, multi-tenant hosted product. It is decomposed into cooperating subsystems:

- **Web Dashboard** – Single-page application: authentication, target management, scan control, live progress, finding triage, report export.
- **Application API** – Stateless HTTP API: validation, authentication and authorisation, persistence, job enqueueing, report generation.
- **Job Queue** – Durable queue decoupling scan requests from execution; retry, priority, cancellation.
- **Scan Engine Workers** – Horizontally scalable processes consuming scan jobs and running the crawl and detection pipeline.
- **Scope Guard** – The safety kernel: every outbound request passes through it (authorisation, scope, rate limits, network blocks).
- **Database** – Durable store for users, targets, scans, findings, evidence, and audit records.

Positioning. WVS belongs to the Dynamic Application Security Testing (DAST) category, comparable to OWASP ZAP, Burp Suite, and Acunetix, rather than to infrastructure vulnerability assessment products such as Nessus or Qualys. It probes an application's own code for defects rather than matching installed software against CVE feeds. Because WVS is a publicly reachable hosted service that originates its own outbound traffic, unlike installed scanners, which can devolve authorisation to their operator, authorisation is enforced as a mandatory safety kernel (C.2, F.2, F.8). Without it, the product would constitute an anonymous, internet-facing attack platform.

2.2 Product Functions

F.1 Account and Access Management

- Register, confirm, authenticate, and manage user accounts and sessions.
- Enforce role-based authorisation and organisation-scoped data isolation.

F.2 Target Management and Authorisation

- Register targets and prove ownership before any scan is permitted.
- Define per-target scope and manage the target lifecycle.

F.3 Scan Configuration and Execution

- Configure scans by profile and run them immediately or on a recurring schedule.
- Queue, execute, pause, resume, and cancel scans with live progress reporting.

F.4 Discovery (Crawling)

- Enumerate in-scope URLs, forms, and parameters within the defined scope.
- Render JavaScript-driven pages to reach client-side routes and API calls.

F.5 Vulnerability Detection

- Execute the passive and safe-active detector set defined for this release.
- Classify each finding by severity, CWE, OWASP category, and confidence.

F.6 Findings Management

- Deduplicate findings and compare them across successive scans of a target.
- Retain sanitised evidence and record triage decisions.

F.7 Reporting and Export

- Generate reports in PDF, HTML, JSON, CSV, and SARIF.
- Serve technical and summary audiences through distinct report templates.

F.8 Safety, Auditing, and Administration

- Enforce rate limits, blocklists, and SSRF protection on all outbound traffic.
- Maintain an immutable audit log and provide administration and a kill switch.

2.3 User Classes

- **Security Analysts** (`ANALYST`): Highly technical users fluent in OWASP categories, HTTP, and exploitation concepts. They need full scan control, access to raw evidence, and a triage workflow.
- **Developers** (`DEVELOPER`): Web developers with limited security specialism. They need clear explanations, concrete remediation guidance, and SARIF export for their existing tooling.
- **Viewers / Stakeholders** (`VIEWER`): Non-technical readers such as managers and clients. They need a posture summary, trends over time, and an exportable PDF report.
- **System Administrators** (`ADMIN`): Operators of the WVS deployment itself. They need user management, quota control, audit review, and system health visibility.

The dashboard accommodates this spread with a layered view: plain-language summary by default, technical evidence on demand (NFR-USE-1).

2.4 Constraints

- **C.1 Non-Destructive Operation** – The scan engine shall never issue a request that mutates target state. Only `GET`, `HEAD`, and `OPTIONS` are permitted by default; `POST` only against forms explicitly marked safe by the user, and never with destructive payloads.
- **C.2 Authorisation Control** – Scanning is permitted only against targets whose ownership is verified per F.2. There shall be no configuration option that disables this.
- **C.3 Schedule and Team** – The system is developed within a single academic term by a student team, ruling out capabilities requiring sustained maintenance (e.g. a self-maintained exploit database).
- **C.4 Hardware Constraints** – The full stack shall run on commodity hardware, a single machine with 8 GB RAM, for demonstration.
- **C.5 Regulatory and Licensing Policies** – All third-party dependencies shall carry OSI-approved licences; no commercially licensed scanning engine shall be embedded.
- **C.6 Implementation Language** – The implementation language shall be TypeScript across client and server (DC-1).
- **C.7 Data Protection** – Personal data shall be limited to what authentication and audit require; no target content shall be retained beyond the evidence retention window (F.6).

2.5 Assumptions and Dependencies

Assumptions:

- Target applications are reachable over the public internet from the deployment environment.
- Users have sufficient administrative control of targets to publish a DNS TXT record or a file under `/.well-known/`.
- Targets respond to well-formed HTTP requests without aggressive bot mitigation; otherwise crawls are incomplete and the scan reports reduced confidence (F.4).
- Public vulnerability advisory data remains freely accessible under its current terms.
- Targets are conventional HTTP/HTTPS applications, not WebSocket-only or gRPC-only services.

Dependencies:

- **D-1** Node.js LTS runtime, v22 or later – runtime.
- **D-2** PostgreSQL 16 or later – runtime.

- **D-3** Redis 7 or later – runtime.
- **D-4** Headless Chromium, via Playwright, for JavaScript rendering – runtime.
- **D-5** OSV advisory API and FIRST EPSS API – external service.
- **D-6** SMTP relay for transactional email – external service.
- **D-7** A registered domain and TLS certificate for the WVS deployment itself – deployment.

---

3. Specific Requirements

3.1 Functional Requirements

One requirement per function group (F.1–F.8, §2.2). **Shall** = mandatory, **should** = recommended, **may** = optional (§1.3).

F.1 Account and Access Management

Description: The system shall support registration with email confirmation, passwords of at least 12 characters and account lockout after repeated failed logins. It shall support the roles `ADMIN`, `ANALYST`, `DEVELOPER`, and `VIEWER`, associate every user with exactly one organisation, and prevent all access to another organisation's data. It should also allow session termination, TOTP two-factor authentication, and account deletion.

Input: Registration details, the emailed confirmation link, login credentials, refresh tokens, TOTP codes, and role and organisation assignments.

Output: Confirmed accounts, access and refresh tokens, account lockouts with notification, and an authorisation decision on every request.

F.2 Target Management and Authorisation

Description: The system shall allow registration of a target by origin and label with an immutable authorisation acknowledgement, and shall require proof of control — a system-issued token published as a DNS TXT record or at a well-known URL — before any scan, re-verified at least every 90 days. It shall refuse any target resolving to a private, loopback, link-local, or cloud-metadata address, or matching the administrator-editable blocklist. It shall allow per-target scope definition, and should allow archiving and permanent deletion.

Input: Target origin and label, the authorisation acknowledgement, the published verification token, the scope definition, and archive or delete requests.

Output: Registered targets in a verified or unverified state, verification outcomes, refusal of prohibited targets with the triggering rule identified, and a stored scope applied to every subsequent scan.

F.3 Scan Configuration and Execution

Description: The system shall provide **Passive**, **Standard**, and **Thorough** profiles and allow per-scan configuration of request rate, concurrency, crawl depth, page count, and request ceilings. Scans shall execute asynchronously through the states `QUEUED`, `RUNNING`, `PAUSED`, `COMPLETED`, `FAILED`, `CANCELLED`, and `ABORTED_SAFETY`, resume from the last checkpoint after a worker failure, and stream progress at least every 2 seconds; cancellation shall halt all outbound requests within 5 seconds. It shall enforce per-organisation

quotas and record the exact profile, detector versions, and configuration of every scan, and should support recurring schedules and completion notifications.

Input: A scan request naming a verified target, a profile, configuration values, control commands, and any recurring schedule.

Output: A scan identifier returned immediately, a live progress stream, a terminal lifecycle state, a reproducibility record, and quota refusals or completion notifications.

F.4 Discovery (Crawling)

Description: The system shall discover in-scope URLs from hyperlinks, form actions, and resource references, rendering JavaScript in a headless browser to reach client-side routes and API calls, and shall parse `robots.txt` and `sitemap.xml`, honouring exclusions by default. It shall enumerate each page's forms and query parameters and deduplicate URLs differing only in session identifiers, cache-busting parameters, or parameter ordering. It shall detect blocking (sustained `403`, `429`, or CAPTCHA responses) and mark affected results reduced-confidence rather than treating absence of findings as absence of vulnerabilities, and shall terminate cleanly at the configured depth, page, and request ceilings, recording which limit bound the scan.

Input: The target origin, the scope definition (F.2), the configured crawl ceilings, `robots.txt` and `sitemap.xml`, and the HTTP responses retrieved.

Output: A deduplicated set of in-scope URLs with a per-page inventory of forms and parameters, any blocking indication with reduced-confidence marking, and a record of the binding limit.

F.5 Vulnerability Detection

Description: The system shall check the pages, forms, and inputs found during discovery for security issues using the detector catalogue in Appendix B. It shall review collected responses without making extra requests and may run safe tests where needed. These tests must never change the target, access private data, run commands, or upload files. Each finding shall explain the issue in plain language and include its severity, confidence, relevant CWE and OWASP categories, affected location, and advice for fixing it. Known component versions should also be checked against public vulnerability information. If a check fails, the system shall record the failure and continue; if its own traffic appears to harm the target, it shall stop the scan immediately.

Input: The pages, forms, parameters, and component-version information found during discovery (F.4), plus public vulnerability information.

Output: Clear, classified findings with severity, confidence, affected location, and remediation advice; a record of any failed checks; and a stopped scan if target safety is affected.

F.6 Findings Management

Description: The system shall deduplicate findings within a scan, listing repeat instances as occurrences, and shall classify each finding against the previous scan of the same target as `NEW`, `PERSISTING`, or `RESOLVED`. Every finding shall be presented with a plain-language description ahead of technical detail, together with severity, CVSS vector, confidence, CWE, OWASP category, affected location, evidence, and remediation guidance. Substantiating request/response pairs shall be retained with credentials, session tokens, and recognised

personal data redacted, and purged after a configurable period defaulting to 90 days. Findings shall support the triage statuses `OPEN`, `CONFIRMED`, `FALSE_POSITIVE`, `ACCEPTED_RISK`, and `RESOLVED`, carried forward across scans, and shall be filterable and sortable.

Input: The raw findings of a completed scan, the previous scan's findings and triage history, triage actions, filter and sort criteria, and the configured retention period.

Output: A deduplicated, classified finding set; full finding views with evidence and remediation; evidence purged at retention expiry; persisted triage decisions inherited by later scans; and filtered, sorted result sets.

F.7 Reporting and Export

Description: The system shall generate reports for any completed scan in PDF, HTML, JSON, CSV, and SARIF v2.1.0, the SARIF consumable by GitHub code scanning without transformation, in at least an **Executive Summary** template (posture, severity distribution, trend, no raw evidence) and a **Technical Report** template (all findings with full evidence and remediation). Every report shall state target identity, scan timestamps, profile, scope, detector versions, and any coverage limitations, including that unauthenticated scanning is not a complete assessment. It should support filtering by severity threshold and triage status, asynchronous generation with notification, and time-limited shareable links.

Input: A completed scan, a template and format selection, optional severity and triage filters, and an optional share expiry.

Output: A downloadable report carrying the mandatory metadata and limitations statement, a notification when generation completes, and any requested shareable link.

F.8 Safety, Auditing, and Administration

Description: The system shall route every outbound scan request through a single Scope Guard enforcing, in order, verification status, scope rules, network blocklist, rate limit, and request ceiling, and shall validate the resolved IP immediately before connection to prevent DNS-rebinding SSRF; a request failing any check shall not be issued. It shall apply a default limit of 10 requests per second per target, identify itself in every request by a `User-Agent` naming the product and a contact URL, and provide an administrator kill switch halting all running scans system-wide. It shall maintain an append-only audit log of authentication, target, scan, override, export, and administrative events, not editable through the application, together with a per-scan ledger of every URL requested. Administrators shall be able to manage accounts and quotas, and should have filtered audit review and a health endpoint.

Input: Every candidate outbound request with its verification, scope, blocklist, rate-limit, and freshly resolved IP context; auditable events as they occur; administrative commands; and audit-log filter criteria.

Output: Outbound requests issued only when every check passes, otherwise suppressed with the failing check recorded; a system-wide halt on kill-switch activation; append-only audit records and a per-scan URL ledger; and applied administrative changes, themselves audited.

3.2 External Interface Requirements

3.2.1 User Interfaces

Description: A web dashboard shall be provided as the sole user interface, covering authentication, target management, scan control, findings triage, reporting, and administration. Views shall be layered, presenting a plain-language summary by default with technical evidence disclosed on demand (NFR-USE-1). Severity shall be conveyed by label and shape as well as colour; all views shall be operable by keyboard alone.

Input:

- Registration, sign-in, and email-confirmation submissions.
- Target registration, scope configuration, and ownership-verification requests.
- Scan profile selection, configuration, and pause/resume/cancel commands.
- Findings filter, sort, and triage actions.
- Report template and format selections.
- Administrative actions: user management, quotas, blocklist edits, kill switch.

Output:

- **Sign in / Register** — authentication and email confirmation.
- **Footer** - contains links to the privacy policy and terms of service and contact details.
- **Dashboard home** — posture across all targets, recent scans, severity distribution.
- **Targets list** — all targets with verification status and last scan result.
- **Target detail** — scope configuration, verification state, scan history, trend.
- **New scan** — profile selection, configuration, quota display.
- **Live scan view** — phase, counters, streaming findings, pause/cancel controls.
- **Findings list** — filterable, sortable, colour- *and* label-coded by severity.
- **Finding detail** — plain-language explanation, evidence, remediation, triage controls.
- **Reports** — template and format selection, generation status, download.
- **Administration** — users, quotas, blocklist, audit log, kill switch.

3.2.2 Hardware Interfaces

Description: The system requires no specialised hardware beyond a standard server or workstation meeting the commodity-hardware constraint (C.4) and interfaces with no hardware device directly.

Input: None.

Output: None.

3.2.3 Software Interfaces

Description: The system shall interface with external services for advisory data, transactional email, name resolution, and JavaScript rendering. Failure of any one of these shall degrade only the feature that depends on it, and the degradation shall be recorded rather than silently absorbed:

- **OSV / EPSS advisory APIs** — map fingerprinted component versions to known CVEs and supply exploitation-probability scores (F.5). On failure the scan proceeds, affected findings are suppressed, and the degradation is recorded on the scan.
- **SMTP relay** — verification and notification email. On failure, messages are queued for retry and account creation is not blocked.

- **Public DNS resolvers** — target resolution and TXT-record ownership verification. On failure the scan fails with a clear diagnostic.
- **Headless Chromium (Playwright)** — JavaScript rendering during the crawl. On failure, pages are crawled as static HTML and the coverage limitation is recorded.

Input:

- OSV and EPSS API responses for queried component versions and CVE identifiers.
- SMTP delivery acknowledgements and failures.
- DNS `A`, `AAAA`, and `TXT` records for target hostnames.
- Rendered DOM, client-side routes, and observed network calls from headless Chromium.

Output:

- Component-version and CVE queries issued to the OSV and EPSS APIs.
- Verification and notification messages handed to the SMTP relay.
- Resolution and TXT-record queries issued to public DNS resolvers.
- Page navigation instructions issued to headless Chromium.
- A recorded degradation notice on any scan affected by an unavailable dependency.

3.2.4 Communications Interfaces

Description: Communication between the client and the API shall use REST over HTTPS with JSON payloads, with live scan updates delivered over an authenticated WebSocket channel. Outbound scan traffic to targets shall use HTTP/1.1 and HTTP/2, rate-limited and scope-enforced (F.8). Internal traffic to the database and queue shall be authenticated and encrypted in transit.

- **Client <-> API** — HTTPS (TLS 1.2+), REST/JSON under `/api/v1`, documented by a generated OpenAPI 3.1 specification. Bearer-token authentication; RFC 9457 problem-details errors; cursor pagination; idempotency keys on mutating endpoints.
- **Client <-> live updates** — WSS at `/ws/scans/{id}`, open to authorised subscribers only.
- **Worker <-> Target** — HTTP/1.1 and HTTP/2 over TCP/TLS.
- **API <-> Database / Queue** — PostgreSQL wire protocol over TLS; Redis protocol, authenticated.

Input:

- Authenticated REST requests carrying JSON bodies and a bearer token.
- WebSocket subscription requests for a specific scan identifier.
- HTTP responses returned by scanned targets.

Output:

- JSON REST responses, with errors in RFC 9457 problem-details form.
- WebSocket events `scan.progress`, `scan.finding`, `scan.status`, and `scan.warning`.
- Rate-limited, scope-enforced HTTP requests issued to targets, each identifying the scanner by `User-Agent` (F.8).

3.3 Non-Functional Requirements

Each requirement is identified as `NFR-CAT-n` (§1.3) and stated so that it can be measured or observed rather than merely asserted.

NFR-PERF-1 Interactive Response Times

Description: The API shall respond to 95% of read requests within 300 ms (99% within 800 ms) under 50 concurrent users; the dashboard shall be interactive within 3 s on a 10 Mbps connection; findings queries shall return within 500 ms for 10,000 historical findings.

NFR-PERF-2 Scan and Report Throughput

Description: A Standard-profile scan of a 200-page target shall complete within 15 minutes at the default rate limit; a scan worker shall not exceed 1 GB resident memory; a 500-finding PDF report shall generate within 60 s.

NFR-SEC-1 Transport and Browser Security Controls

Description: All client–server communication shall use TLS 1.2+ with HSTS (`max-age` >= 1 year); the app shall set a restrictive CSP, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, and `Permissions-Policy`; session cookies shall be `HttpOnly`, `Secure`, `SameSite=Strict`.

NFR-SEC-2 Input Validation and Object-Level Authorisation

Description: All input crossing a trust boundary shall be validated against an explicit schema; all database access shall use parameterised queries or an ORM; every endpoint shall enforce object-level authorisation verifying the principal's organisation owns the resource.

NFR-SEC-4 Endpoint Rate Limiting and Error Disclosure

Description: Authentication, registration, target verification, and scan initiation endpoints shall be rate-limited; error responses shall not disclose stack traces, framework versions, internal hostnames, or SQL fragments.

NFR-USE-1 Self-Service Onboarding and Layered Explanation

Description: A first-time user shall be able to register a target, verify ownership, and complete a scan using only in-product guidance; every finding shall be explained in plain language before technical detail, with evidence progressively disclosed.

NFR-MAINT-1 Testability, Code Quality, and Observability

Description: Automated test coverage shall be >= 80% of statements for the scan engine and API, with every detector tested against a purpose-built vulnerable fixture application; linting and type-checking shall pass with zero errors as a condition of merge; the system shall emit structured logs with correlation IDs across API, queue, and worker; schema changes shall use versioned, reversible migrations.

NFR-SCAL-1 Horizontal Scalability

Description: The API tier shall be stateless and horizontally scalable; scan throughput shall scale by adding workers without code changes; the system shall sustain 20 concurrent scans on 4 workers.

NFR-PORT-1 Platform and Browser Portability

Description: The complete system shall run via a single container-orchestration definition on Linux, macOS, and Windows (WSL2) hosts; the dashboard shall support the current and previous major versions of Chrome, Firefox, Safari, and Edge; the engine shall handle HTTP/1.1 and HTTP/2 and correctly process declared response encodings.

NFR-QUAL-1 Detector Accuracy Targets

Description: Against the project's benchmark fixture suite (every vulnerability deliberately introduced and catalogued as ground truth), the system shall detect >= 85% of in-catalogue vulnerabilities and report <= 15% false positives among `HIGH/CRITICAL` findings. Results shall be measured and published in the project report, including where they fall short.

3.4 Design Constraints

DC-1 Implementation Language: TypeScript in strict mode shall be used for all first-party code, client and server. Introducing a second implementation language is out of scope for the current design (C.6).

DC-2 Application Frameworks: The server shall be built on NestJS and the client on React with Vite. Substituting either framework is out of scope for the current design.

DC-3 Persistence: Data shall be persisted in PostgreSQL, accessed exclusively through the Prisma ORM. Direct SQL outside the ORM's parameterised interface is not permitted (NFR-SEC-2).

DC-4 Asynchronous Work: Scans and report generation shall be queued through BullMQ backed by Redis; no long-running work shall execute inside a request handler.

DC-5 JavaScript Rendering: Client-side rendering during discovery shall use Playwright driving headless Chromium. The engine shall remain functional, with reduced coverage, where rendering is unavailable (§3.2.3).

DC-6 API Surface: The API shall be REST with an OpenAPI 3.1 specification generated from the implementation rather than maintained by hand, and live dashboard updates shall use WebSocket transport.

DC-7 Packaging: The system shall be packaged as containers with a single orchestration definition serving both local development and demonstration deployment, and shall run within the commodity-hardware limit of C.4.

DC-8 Detector Authoring: Detectors shall declare their metadata (identifier, version, category, CWE, default severity, passive/active) as data. Declarative detectors shall be authored in YAML and validated against a published JSON Schema, and every template-issued request shall pass through the Scope Guard (F.8). No template may bypass scope, rate-limit, or blocklist enforcement.

DC-9 Data Model Integrity: Findings shall be immutable once written, with triage state recorded as separate versioned associations. Audit records shall be append-only at the database privilege level, evidence shall be stored separately from finding metadata to permit independent purging (F.6), and all timestamps shall be stored in UTC.

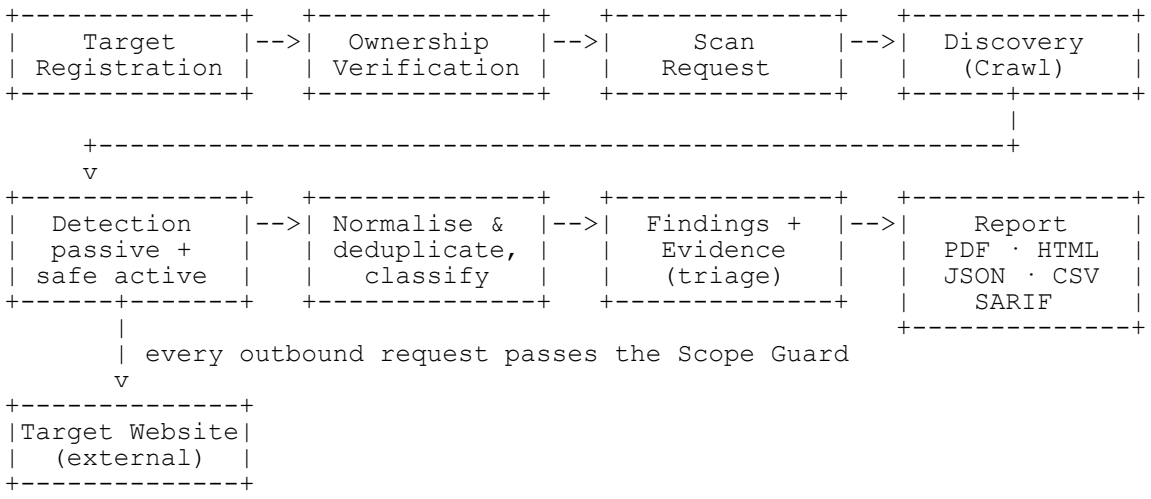
---

4. Appendices

Appendix A: Block Diagram

A.1 Scan Workflow

The block diagram below summarises the scan workflow. A user registers and verifies a target before requesting an in-scope scan. Discovery identifies reachable URLs and inputs, then passive and safe-active detectors assess the target through the Scope Guard. Findings are normalised and retained with sanitised evidence before a report is produced.



Appendix B: Detector Catalogue

B.1 Passive Detectors

Each entry reads: **ID** name — CWE, OWASP 2021 category, default severity, priority.

- **P-01** Missing or weak Content-Security-Policy — CWE-693, A05, Medium, M.
- **P-02** Missing HTTP Strict-Transport-Security — CWE-319, A02, Medium, M.
- **P-03** Missing `X-Content-Type-Options: nosniff` — CWE-693, A05, Low, M.
- **P-04** Missing or permissive frame-ancestors / X-Frame-Options — CWE-1021, A05, Medium, M.
- **P-05** Missing or overly permissive Referrer-Policy — CWE-200, A01, Low, M.
- **P-06** Missing Permissions-Policy — CWE-693, A05, Info, S.
- **P-07** Cookie without `Secure` attribute — CWE-614, A05, Medium, M.
- **P-08** Cookie without `HttpOnly` attribute — CWE-1004, A05, Medium, M.
- **P-09** Cookie without or with weak `SameSite` — CWE-1275, A01, Low, M.
- **P-10** Cookie scoped too broadly (parent domain) — CWE-565, A01, Low, S.
- **P-11** Deprecated TLS protocol offered (TLS 1.0 / 1.1 / SSLv3) — CWE-327, A02, High, M.
- **P-12** Weak cipher suite offered — CWE-327, A02, High, M.
- **P-13** Certificate expired, not yet valid, or expiring within 30 days — CWE-324, A02, High, M.
- **P-14** Certificate hostname mismatch — CWE-297, A07, High, M.
- **P-15** Self-signed or untrusted certificate chain — CWE-295, A07, High, M.
- **P-16** Weak certificate signature algorithm or key size — CWE-327, A02, Medium, S.
- **P-17** Server / framework version disclosure in headers — CWE-200, A05, Low, M.

- **P-18** Technology and component fingerprinting — no CWE/OWASP mapping, Info, M.
- **P-19** Known-vulnerable component version (OSV/CVE correlation) — CWE-1104, A06, varies by CVE, S.
- **P-20** Verbose error page or stack trace disclosure — CWE-209, A05, Medium, M.
- **P-21** Exposed version control directory (`.git`, `.svn`, `.hg`) — CWE-527, A05, Critical, M.
- **P-22** Exposed environment or configuration file (`.env`, `web.config`) — CWE-538, A05, Critical, M.
- **P-23** Directory listing enabled — CWE-548, A05, Medium, M.
- **P-24** Backup or temporary file exposure (`.bak`, `~`, `.old`) — CWE-530, A05, High, S.
- **P-25** Mixed active content on HTTPS page — CWE-311, A02, Medium, M.
- **P-26** External script without Subresource Integrity — CWE-353, A08, Low, S.
- **P-27** Secret or credential pattern in response body — CWE-540, A05, Critical, M.
- **P-28** Email or personal data disclosure in response — CWE-200, A01, Low, S.
- **P-29** Sensitive page cacheable by intermediaries — CWE-525, A04, Low, S.
- **P-30** Missing or malformed `security.txt` (RFC 9116) — no CWE/OWASP mapping, Info, C.
- **P-31** Autocomplete enabled on password or sensitive field — CWE-522, A07, Low, C.

B.2 Safe Active Detectors

All detectors in this list are restricted by F.5: idempotent methods, benign marker payloads, no data modification, no command execution, and no file writing. Each entry reads: **ID** name — detection technique, CWE, OWASP 2021 category, default severity, priority.

- **A-01** Reflected Cross-Site Scripting — benign marker reflected unencoded into an executable context. CWE-79, A03, High, M.
- **A-02** SQL Injection (error-based) — syntax-breaking characters matched against database error signatures. CWE-89, A03, Critical, M.
- **A-03** SQL Injection (boolean-based) — responses to logically true and false conditions compared. CWE-89, A03, Critical, S.
- **A-04** Open Redirect — redirect parameter pointed at a sentinel host; `Location` confirmed. CWE-601, A01, Medium, M.
- **A-05** CORS misconfiguration — varied `Origin` values; reflection, `null`, or credentialed wildcard detected. CWE-942, A05, High, M.
- **A-06** Clickjacking — page confirmed framable without frame-ancestor restrictions. CWE-1021, A05, Medium, M.
- **A-07** Dangerous HTTP methods enabled — `OPTIONS` enumeration; `TRACE`/`PUT`/`DELETE` verified non-destructively. CWE-650, A05, Medium, M.
- **A-08** Host header injection — alternate `Host` reflected unvalidated into links or redirects. CWE-644, A03, Medium, S.
- **A-09** Sensitive file and directory enumeration — bounded, rate-limited wordlist of administrative and backup paths. CWE-538, A05, varies, M.
- **A-10** Missing anti-CSRF token on state-changing form — forms analysed structurally for a token field and its unpredictability. CWE-352, A01, Medium, S.
- **A-11** Path traversal (read-only probe) — traversal sequences matched against read-only file signatures. CWE-22, A01, High, S.
- **A-12** Server-side template injection (detection only) — arithmetic marker expression evaluated; no further exploitation. CWE-1336, A03, High, C.
- **A-13** Unauthenticated access to administrative interface — known admin paths classified as gated or open. CWE-306, A01, High, S.

- **A-14** Improper HTTPS redirection — plaintext origin verified to redirect to HTTPS. CWE-319, A02, Medium, M.

Version History

- **1.0** (31 July 2026) – Initial SRS draft.